

A BRIEF COOKBOOK OF CODING

Contents

1	Data Structures	3
1.1	Linear Structures	3
1.1.1	Hashtable	3
1.1.2	Stack	5
1.1.3	Queue	5
1.2	Tree	6
1.2.1	Heap	6
1.2.2	Balanced Search Tree	7
1.2.3	Fenwick Tree	10
1.2.4	Segment Tree	13
1.3	Graph	15
1.3.1	Disjoint Set Union	15
1.3.2	Bipartite Graphs	18
2	Algorithms	23
2.1	Handy Tricks	23
2.1.1	Prime Sieve	23
2.1.2	KMP Algorithm	23
2.1.3	Quick Power	24
2.2	Search	25
2.2.1	DFS	25
2.2.2	BFS	26
2.2.3	Binary Search	26
2.3	Divide and Conquer	27
2.4	Dynamic Programming	29
2.5	Graph Theory	36
2.5.1	Minimum Spanning Tree	36
2.5.2	Single Source Shortest Path	37
2.5.3	Strongly Connected Components	39
2.5.4	Topological Sort	40

Do you know why the table of contents can't be part of the table of contents? Because of self-reference! And that's pretty much the core idea behind this coding cookbook.

For a better coding experience — let's use Python in this cookbook. This isn't to criticize C++ or Java, but Python code is usually much easier to express ideas (while still offering good performance for most cases). We do not delve into low-level implementations in depth, as they are less important than their use within higher-level ideas.

I used to code primarily in C++. Note that data structures and algorithms are far more important than any specific programming language. Try solving problems on Codeforces, LeetCode or Luogu — or any coding platform you prefer!

rainrzk
Aug 2, 2026

1 Data Structures

1.1 Linear Structures

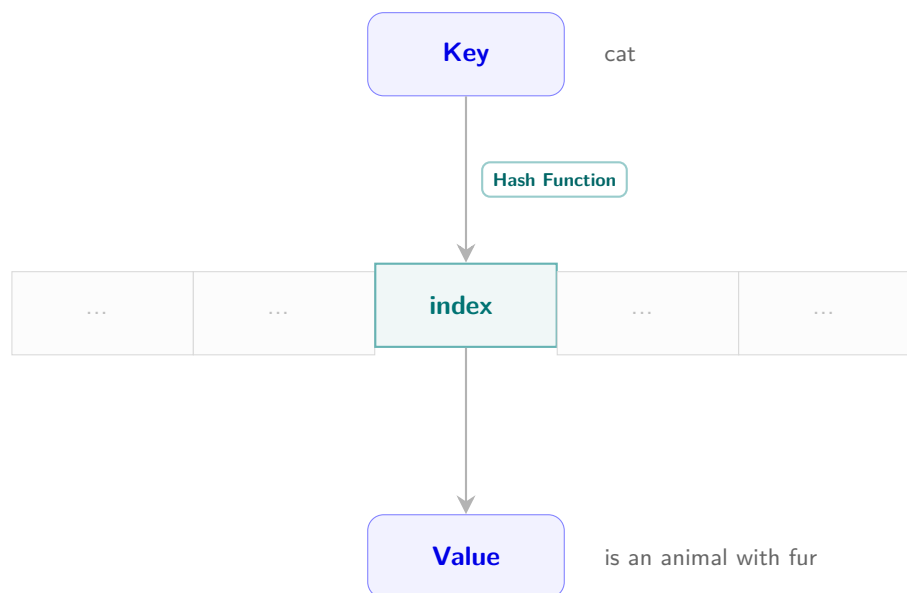
1.1.1 Hashtable

Hashtable is the *first* linear structure I'm going to talk about. We simply use

```
1 m = {}
```

to create a *map* / *dict*.

A hashing process is defined by a function that maps a key to the index where its value is stored. In the best case, the time complexity is $\mathcal{O}(1)$.



Q1 (Bucket Sort ★★).

Given n points on a 2D plane where `points[i] = [xi, yi]`, return the widest vertical area between two points such that no points are inside the area.

A **vertical area** is an area of fixed-width extending infinitely along the y -axis (i.e. infinite height). The widest vertical area is the one with the maximum width.

Note that points on the edge of a vertical area are not considered included in the area.

Take some time to think about it before looking at the solution —

Intuitively, we could solve this in $\mathcal{O}(n \log n)$ time by sorting all x -coordinates and scanning the adjacent intervals to find the largest one.

However, there is a $\mathcal{O}(n)$ approach using hashtable.

Let's partition the x -coordinates into several equal-sized *buckets*. To guarantee an optimal distribution, how do we determine the exact width of each bucket? Here is the mathematical part:

$$\text{bucket_size} = \max\left(1, \left\lfloor \frac{\text{max_x} - \text{min_x}}{n - 1} \right\rfloor\right).$$

I claim that at least one bucket will be completely empty. Consequently, the maximum gap must span across one or more empty buckets. Because of this, we don't need to care about the points inside a single bucket — we simply need to record the min and max values of each one!

```

1  class Solution:
2      def maxWidthOfVerticalArea(self, points: List[List[int]]) -> int:
3          xs = [p[0] for p in points]
4          min_x, max_x = min(xs), max(xs)
5          gap = max(1, (max_x - min_x) // (len(xs) - 1))
6          bucket_num = (max_x - min_x) // gap + 1
7          buckets = [[inf, -inf] for _ in range(bucket_num)]
8
9          for x in xs:
10             idx = (x - min_x) // gap
11             buckets[idx][0] = min(buckets[idx][0], x)
12             buckets[idx][1] = max(buckets[idx][1], x)
13
14             res, prev = 0, min_x
15
16             for b in buckets:
17                 if b[0] == inf:
18                     continue
19                 res = max(res, b[0] - prev)
20                 prev = b[1]
21
22             return res

```

This method is called **Bucket Sort**.

Q2 (Three Equal Sums ★).

Given an array with total sum K , return the number of ways to partition the array into three contiguous subarrays such that the sums of the three subarrays are equal.

Can we split the array into three parts such that each part has a sum equal to $\frac{K}{3}$? Furthermore, can we count how many times the sum of the first subarray equals $\frac{K}{3}$? The answer is *yes* in $\mathcal{O}(n)$ in one pass, through a hashtable.

```

1  class Solution:
2      def countThreeSums(self, arr: list[int], K: int) -> int:
3          if sum(arr) != 3 * K:
4              return 0
5          ways = 0
6          count_k = 0
7          current_sum = 0
8
9          for i in range(len(arr) - 1):

```

```
10         current_sum += arr[i]
11         if current_sum == 2 * K:
12             ways += count_k
13         if current_sum == K:
14             count_k += 1
15
16     return ways
```

1.1.2 Stack

Stack is an implementation of **LIFO**. We simply use

```
1 s = []
```

to create a *list*.

We pick the top element by

```
1 top = s.pop() or top = s[-1]
```

and by

```
1 s.append(a)
```

we append a new element *a*.

1.1.3 Queue

Queue is an implementation of **FIFO**. Unfortunately, Python does not have a built-in queue — we use *deque*

```
1 q = deque()
```

to create a queue.

We pick the front element by

```
1 front = q.popleft() or front = q[0]
```

and by

```
1 q.append(b)
```

we append a new element *b*.

Note: Deques are not *linked lists*, and we will not discuss linked lists further, since they are not really used in practice and can usually be replaced by better alternatives.

1.2 Tree

1.2.1 Heap

Heap is an implementation of **Priority Queue** based on lists. Python only supports *min-heaps* (where root is the smallest element).

We create a min-heap by

```
1 h1 = []
2 heapq.heappush(h1, a)
3 h2 = [b, c]
4 heapq.heapify(h2)
```

and by

```
1 heapq.heappop(h2)
```

we pop the root.

Q3 (Meeting Rooms ★).

Given an array of meeting time intervals `intervals` where `intervals[i] = [starti, endi]`, return the minimum number of conference rooms required.

We **ONLY** care about the finishing time of each meeting — that is, for every new meeting, let's compare it with the earliest finishing time — if it is smaller, we append it; otherwise, we pop and append it. This is exactly a heap process, which is $\mathcal{O}(n \log n)$.

```
1 class Solution:
2     def minMeetingRooms(self, intervals: List[List[int]]) -> int:
3         intervals.sort(key=lambda x: x[0])
4         room = []
5         heapq.heappush(room, intervals[0][1])
6         ans = 1
7
8         for x in intervals[1:]:
9             while room and x[0] >= room[0]: heapq.heappop(room)
10            heapq.heappush(room, x[1])
11            ans = max(ans, len(room))
12
13        return ans
```

Q4 (Find Median from Data Stream ★★★).

The median is the middle value in an ordered integer list. If the size of the list is even, there is no middle value, and the median is the mean of the two middle values. Implement the MedianFinder class:

- `MedianFinder()` initializes the `MedianFinder` object.
 - `void addNum(int num)` adds the integer `num` from the data stream to the data structure.
 - `double findMedian()` returns the median of all elements so far. Answers within 10^{-5} of the actual answer will be accepted.
-

Quite tricky, isn't it? We can't afford to sort the data every time we insert a new element — so we need a structure that maintains order — that's where a dual-heap approach comes in.

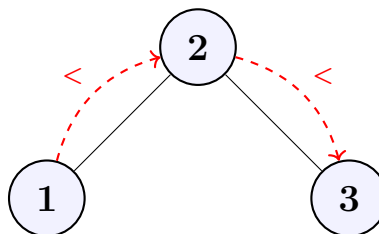
We create a min-heap for the larger part and a max-heap for the smaller part (by taking negatives in a min-heap). Every time a new element comes in, we compare it with the top of the min-heap, and after heapifying we rebalance the two heaps to keep their sizes close to each other (which is the definition of maintaining the median). Thus, this yields an $\mathcal{O}(n \log n)$ solution.

```
1  class MedianFinder:
2      def __init__(self):
3          self.lowHeap = [1e5]
4          self.highHeap = [1e5]
5          lowSize, highSize = 0, 0
6
7      def addNum(self, num: int) -> None:
8          low = -self.lowHeap[0]
9          high = self.highHeap[0]
10         if num <= low: heapq.heappush(self.lowHeap, -num)
11         else: heapq.heappush(self.highHeap, num)
12         self.lowSize = len(self.lowHeap)
13         self.highSize = len(self.highHeap)
14         if self.lowSize - self.highSize > 1:
15             num = -heapq.heappop(self.lowHeap)
16             heapq.heappush(self.highHeap, num)
17         elif self.highSize - self.lowSize > 1:
18             num = heapq.heappop(self.highHeap)
19             heapq.heappush(self.lowHeap, -num)
20
21     def findMedian(self) -> float:
22         if self.lowSize - self.highSize == 1: return -self.lowHeap[0]
23         elif self.highSize - self.lowSize == 1: return self.highHeap[0]
24         else: return (-self.lowHeap[0] + self.highHeap[0]) / 2
```

1.2.2 Balanced Search Tree

A *Binary Search Tree (BST)* is a binary tree satisfying, for every node that, $\text{left subtree} < \text{node} < \text{right subtree}$. It's related to *Binary Search*.

If we compare three traversal methods (*preorder*: root-left-right, *inorder*: left-root-right, *postorder*: left-right-root), binary search trees produce an sequence in inorder traversal. Inorder traversal is equivalent to projecting all nodes onto the bottom line from left to right.



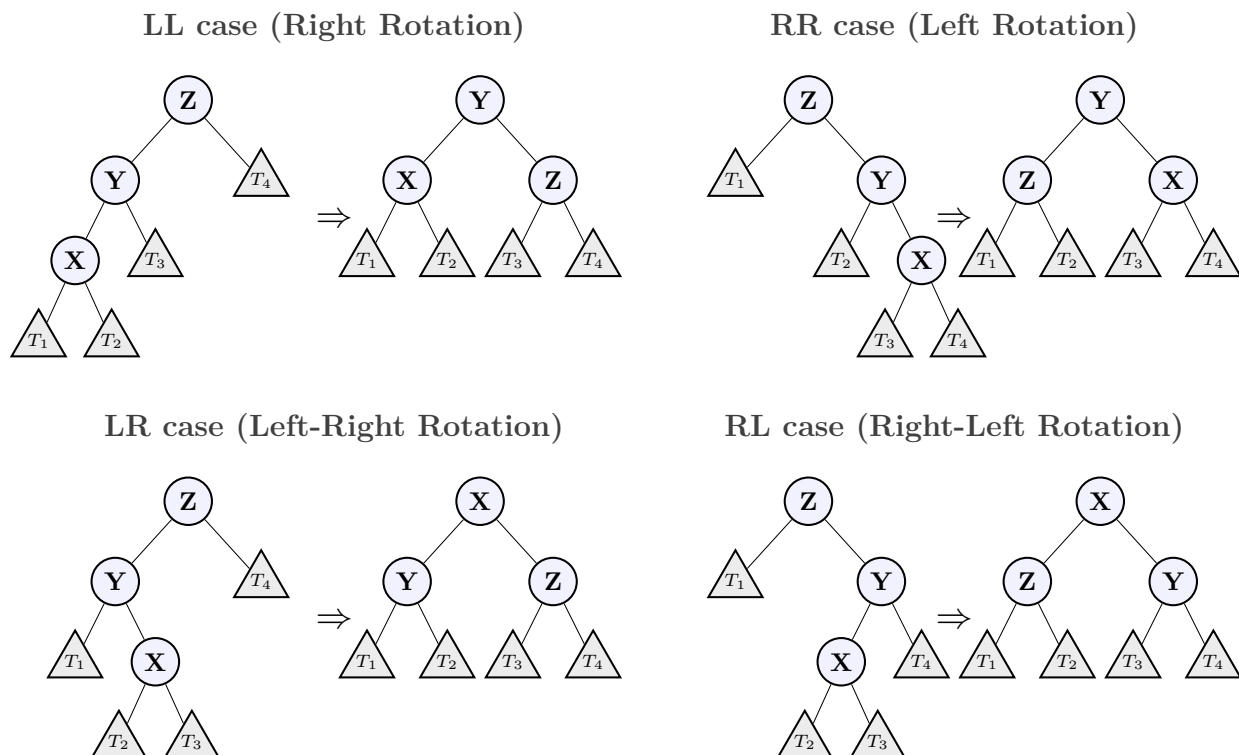
An insertion always fits into the correct position without violating the inorder order, and a deletion always replaces an internal node with the minimum node in its right subtree to a leaf deletion.

A **Balanced Search Tree** is a search tree whose subtree heights are balanced to some extent (could be *strictly* or *unstrictly*).

a) AVL Tree

An **AVL Tree** is a balanced BST where the height difference between the left and right subtrees of any node is *strictly* at most 1.

We introduce four types of *rotation*. Denote the current node by X, the parent by Y, and the grandparent by Z.



Note: An insertion takes at most two rotations (can restore the subtree to its original height, as seen in four cases), but a deletion may take $\mathcal{O}(\log n)$ rotations (e.g. deleting an inner leaf can trigger a rotation, after which the opposite subtree of an ancestor can become two levels taller than the current subtree).

b) Red-Black Tree

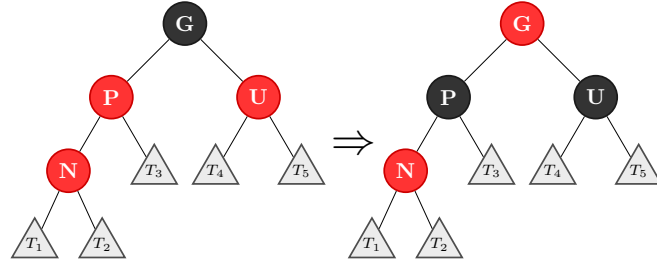
Why am I highlighting this? Because it refers to `std::map` and `std::set`, which are common in C++ but do not exist in Python. **Red-Black Trees** outperform AVL trees in many ways.

We define the root and the leaves to be *black*, and every *red* node must have black children. The core is that every path with the same **black height** (ending at a black node) must have the same number of black nodes.

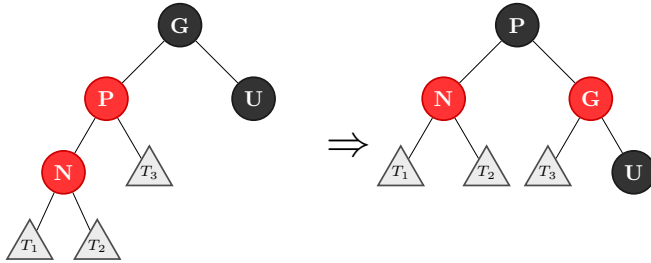
Therefore, the longest path cannot be more than twice the length of the shortest path. Since any inserted node must be red, rotation only occurs when the inserted node's parent is red.

We introduce *color flip*. Denote the current node by N, the parent by P, the uncle by U, and the grandparent by G.

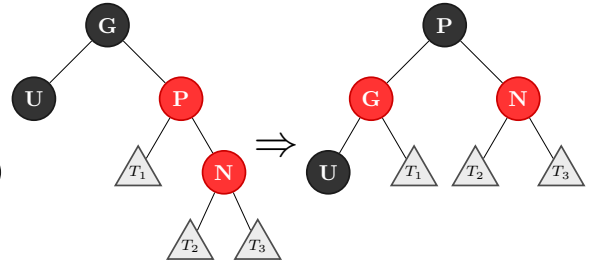
Color Flip



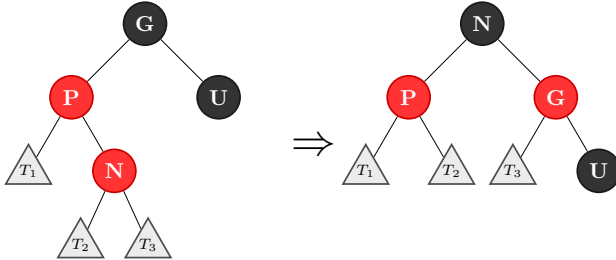
LL case (Right Rotation + Color Flip)



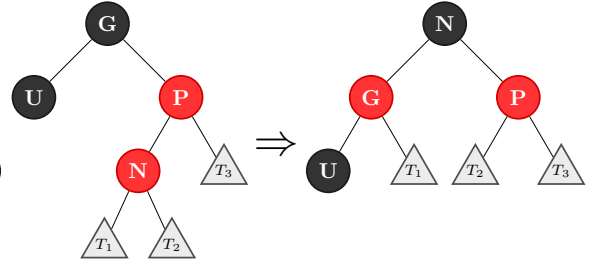
RR case (Left Rotation + Color Flip)



LR case (Left-Right Rotation + Color Flip)



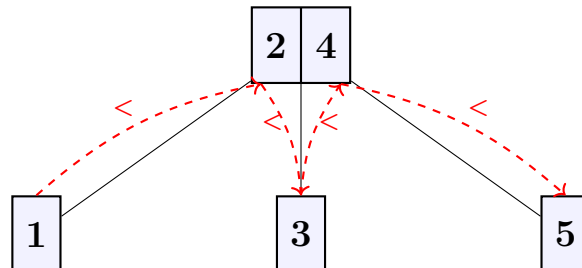
RL case (Right-Left Rotation + Color Flip)



Note: An insertion takes at most two rotations (can restore the subtree to its original black height, as seen in five cases), and a deletion takes at most three rotations (e.g. deleting a black node whose sibling is red can first require a left rotation, followed by an RL case that requires two rotations).

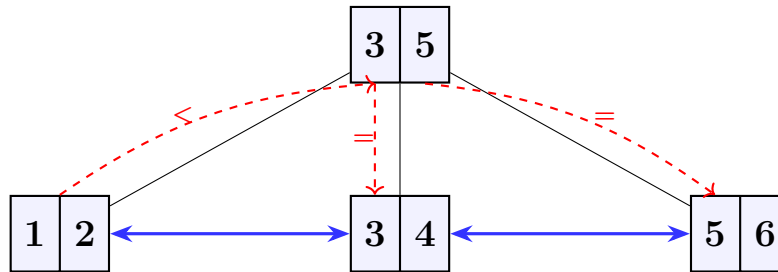
c) B-Tree

A **B-Tree** is a strictly balanced *Multiway Search Tree (MST)*. The *order* m denotes the maximum number of children per node, therefore each node can store at most $m-1$ keys (shown below for $m = 3$). Only when the root splits does the height increase by one, otherwise it remains unchanged.



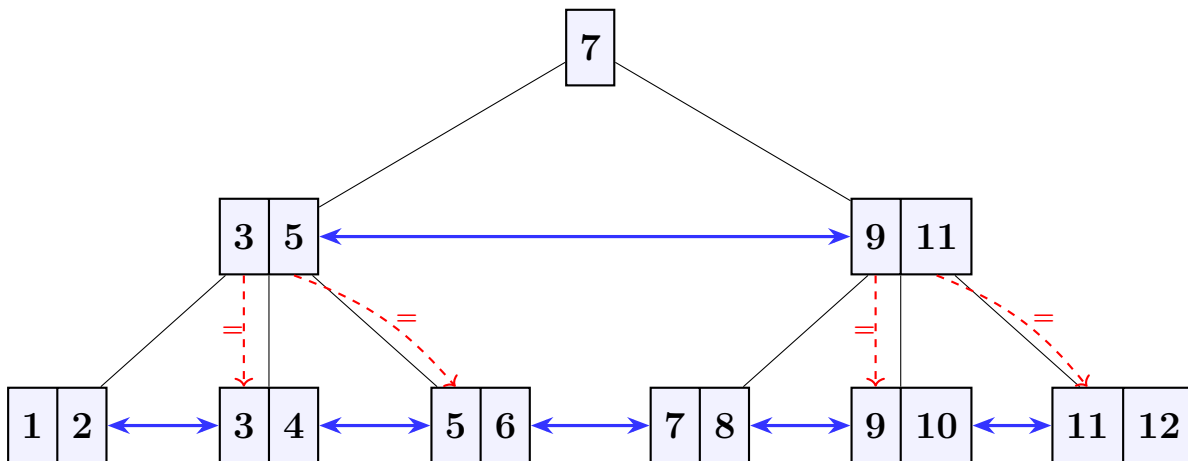
d) B+ Tree

A **B+ Tree** is also a strictly balanced MST. Only the leaves store data — the internal nodes are purely indices (not real data). The leaves at the bottom are stored in a *doubly linked list*.



e) B* Tree

A **B* Tree** is a B+ Tree. Not only are the leaves at the bottom stored in a doubly linked list, but so are the nodes at every level. If a node is full, it *gives* some of its keys to a sibling before splitting.



1.2.3 Fenwick Tree

A **Fenwick Tree** is also called a *Binary Indexed Tree (BIT)*. Before we start with it, we shall first introduce *prefix sums* and *suffix sums*.

Q5 (Subarray Sum Equals K ★).

Given an array of integers `nums` and an integer `K`, return the total number of subarrays whose sum equals to `K`. A subarray is a contiguous non-empty sequence of elements within an array.

A subarray sum is the difference between two prefix sums. From this idea, we only need to store all prefix sums in a hashmap. In this way, we can solve the problem in a single $\mathcal{O}(n)$ pass.

```
1 class Solution:
2     def subarraySum(self, nums: List[int], k: int) -> int:
3         m = {}
4         ans = 0
5         sum = 0
6
```

```

7         for num in nums:
8             if sum in m:
9                 m[sum] += 1
10            else:
11                m[sum] = 1
12            sum += num
13            if sum - k in m:
14                ans += m[sum - k]
15
16        return ans

```

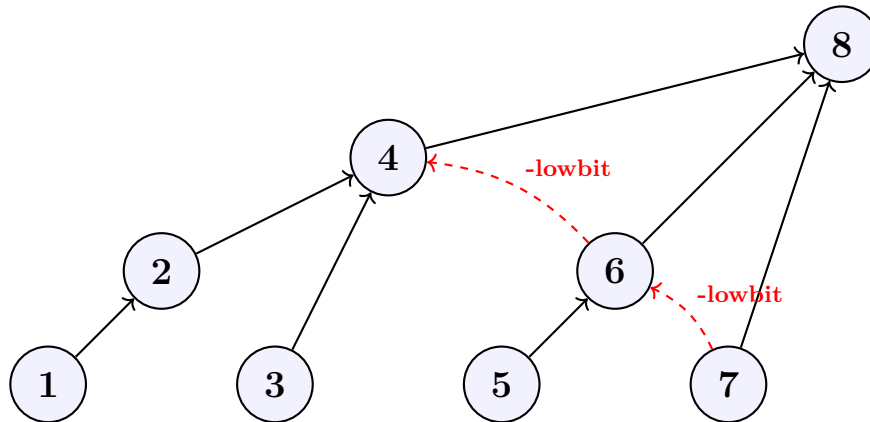
Q6 (Count of Smaller Numbers After Self ★★★).

Given an integer array `nums`, return an integer array `counts` where `counts[i]` is the number of smaller elements to the right of `nums[i]`.

Intuitively, we can start from the right side, adding one to every key in the hashtable that is greater than the current number — however this is **IMPOSSIBLE** because the range of numbers is too large, and each update would take $\mathcal{O}(n)$ time!

In fact, the first problem is easy to solve using *discretization*. How can we reduce the second problem to $\mathcal{O}(\log n)$? Hint: we need to add one to a range at each step.

Now it is time to introduce Fenwick Trees.



This is a classical prefix Fenwick Tree using 1-based indexing. So please don't forget to add 1, since arrays start counting from 0.

There is a *lowbit* trick called `lowbit(i) = i & (-i)`, which extracts the least significant set bit. During an element update we recursively apply `i += lowbit(i)`, whereas during a range query we recursively apply `i -= lowbit(i)`.

```

1  class Solution:
2      def lowbit(self, x: int) -> int:
3          return x & (-x)
4
5      def add(self, index: int, delta: int) -> None:
6          i = index + 1
7          while i <= self.N:
8              self.fenwick[i] += delta

```

```

9         i += self.lowbit(i)
10
11     def query(self, index: int) -> int:
12         val = 0
13         i = index + 1
14
15         while i > 0:
16             val += self.fenwick[i]
17             i -= self.lowbit(i)
18
19         return val
20
21     def countSmaller(self, nums: list[int]) -> list[int]:
22         n = len(nums)
23         ans = [0] * n
24         discrete_nums = sorted(list(set(nums)))
25         self.N = len(discrete_nums)
26         self.fenwick = [0] * (self.N + 1)
27
28         for i in range(n - 1, -1, -1):
29             rank = bisect.bisect_left(discrete_nums, nums[i]) + 1
30             ans[i] = self.query(rank - 1)
31             self.add(rank, 1)
32
33         return ans

```

The situation above involves adding new elements to a Fenwick Tree while querying a range sum at the same time. But what if we only need to convert a normal array into a Fenwick Tree? Instead of updating each element in $\mathcal{O}(n \log n)$ time, we can build it in $\mathcal{O}(n)$ time.

```

1  for i in range(1, n + 1):
2      par = i + lowbit(i)
3      if par <= n:
4          fenwick[par] += fenwick[i]

```

This is the so-called **Contribution Technique**.

In fact, Fenwick Trees can also be used for range additions. See the following problem.

Q7 (Interval Fenwick ★★★★★).

Design a data structure that efficiently supports range updates and prefix sum queries.

Implement the `IntervalFenwick` class:

- `IntervalFenwick(int n)` initializes the data structure with an array of size n containing all 0s.
 - `void updateRange(int left, int right, int value)` adds `value` to all elements in the inclusive range `[left, right]`.
 - `int queryRange(int index)` returns the sum of all elements in the inclusive prefix range `[1, index]`.
-

How can we *transform* range updates into point updates?

Use *Abel's Transformation* as a tool! Let's say we use $A[i]$ as the original array. We take its derivative

(technically, the difference), the sum query becomes:

$$\sum_{i=1}^x A[i] \cdot 1 = x \cdot A[x] - \sum_{i=1}^{x-1} i \Delta A[i] = x \left(\sum_{i=1}^{x-1} \Delta A[i] + A[1] \right) - \sum_{i=1}^{x-1} i \Delta A[i].$$

Simplify this using $\Delta A[i] = D[i + 1]$:

$$x \left(\sum_{i=1}^{x-1} \Delta A[i] + A[1] \right) - \sum_{i=1}^{x-1} i \Delta A[i] = (x + 1) \sum_{i=1}^x D[i] - \sum_{i=1}^x i D[i].$$

We only need to update the two range boundaries rather than the whole range. Moreover, for a range query, we calculate the prefix sum by querying $D[i]$ and $iD[i]$ which represent two Fenwick Trees.

```

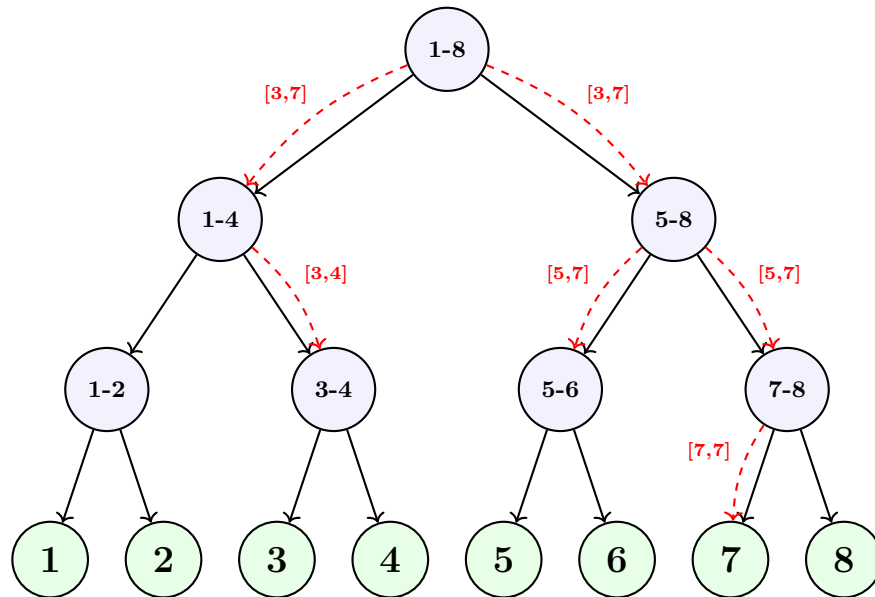
1  class IntervalFenwick:
2      def __init__(self, n: int):
3          self.n = n
4          self.d1 = [0] * (n + 2)
5          self.d2 = [0] * (n + 2)
6
7      def lowbit(self, x: int) -> int:
8          return x & (-x)
9
10     def add(self, tree: list[int], index: int, delta: int) -> None:
11         while index <= self.n + 1:
12             tree[index] += delta
13             index += self.lowbit(index)
14
15     def query(self, tree: list[int], index: int) -> int:
16         res = 0
17
18         while index > 0:
19             res += tree[index]
20             index -= self.lowbit(index)
21
22         return res
23
24     def updateRange(self, left: int, right: int, value: int) -> None:
25         self.add(self.d1, left, value)
26         self.add(self.d1, right + 1, -value)
27         self.add(self.d2, left, left * value)
28         self.add(self.d2, right + 1, (right + 1) * (-value))
29
30     def queryRange(self, index: int) -> int:
31         sum_d1 = self.query(self.d1, index)
32         sum_d2 = self.query(self.d2, index)
33         return (index + 1) * sum_d1 - sum_d2

```

1.2.4 Segment Tree

Segment Trees are slightly slower than Fenwick Trees in terms of constant factors but they are much more general, especially for range updates (see propagation below). A segment tree is a balanced binary tree.

Why do we call them segment trees? Because each node represents an interval of indices, or, a segment of the index range.



If a node is completely included in the range to be updated, it immediately stops propagating and adds a *lazy tag* to itself. Only when another operation (either an update or a query) reaches this node is the lazy tag pushed down to its children.

This is called **Lazy Algorithm** (last but not least, lazy algorithms are applied to, but not limited to Segment Trees, DSUs, Dijkstra's Algorithm, etc.).

```

1  class SegmentTree:
2      def __init__(self, n: int):
3          self.segment = [0] * (4 * n)
4          self.mark = [0] * (4 * n)
5
6      def pushdown(self, p: int, cl: int, cr: int) -> None:
7          mid = (cl + cr) // 2
8          left_child = 2 * p
9          right_child = 2 * p + 1
10         self.mark[left_child] += self.mark[p]
11         self.segment[left_child] += self.mark[p] * (mid - cl + 1)
12         self.mark[right_child] += self.mark[p]
13         self.segment[right_child] += self.mark[p] * (cr - mid)
14         self.mark[p] = 0
15
16     def query(self, l: int, r: int, p: int, cl: int, cr: int) -> int:
17         if cl > r or cr < l:
18             return 0
19         if cl >= l and cr <= r:
20             return self.segment[p]
21         self.pushdown(p, cl, cr)
22         mid = (cl + cr) // 2
23         return self.query(l, r, 2 * p, cl, mid) + self.query(l, r, 2 * p + 1, mid
24 + 1, cr)

```

```

25     def update(self, l: int, r: int, val: int, p: int, cl: int, cr: int) -> None:
26         if cr < l or cl > r:
27             return
28         if cl >= l and cr <= r:
29             self.segment[p] += val * (cr - cl + 1)
30             self.mark[p] += val
31             return
32         mid = (cl + cr) // 2
33         self.pushdown(p, cl, cr)
34         self.update(l, r, val, 2 * p, cl, mid)
35         self.update(l, r, val, 2 * p + 1, mid + 1, cr)
36         self.segment[p] = self.segment[2 * p] + self.segment[2 * p + 1]

```

We use an $\mathcal{O}(2n)$ way to build a segment tree, and this is a very typical example of *Divide and Conquer*:

```

1     def build(self, l: int, r: int, p: int) -> None:
2         if l == r:
3             self.segment[p] = self.nums[l - 1]
4             return
5         mid = (l + r) // 2
6         self.build(l, mid, 2 * p)
7         self.build(mid + 1, r, 2 * p + 1)
8         self.segment[p] = self.segment[2 * p] + self.segment[2 * p + 1]

```

Note: Hmm, don't you think we can skip the pushdown for the update process if the update operation is *commutative*? We can definitely implement this with a better constant factor by a *Permanent Tags* trick!

If we lean more into an abstract algebra view, let's denote the content in the nodes as maintaining an operation $\oplus$, forming a *monoid* with *associativity* $(G, \oplus)$, and the lazy tag as representing a function f , which also forms a monoid with associativity $(F, \circ)$. The requirement for a segment tree to hold is equivalent to:

$$f(a \oplus b) = f(a) \oplus f(b)$$

which is an *endomorphism*. If the lazy tag $(F, \circ)$ is commutative, it is an *Abelian Monoid* that can apply the permanent tags trick.

1.3 Graph

1.3.1 Disjoint Set Union

Since adjacency lists and adjacency matrices being too straightforward to write much about (and they'll occur in many questions later), I'm going to start with *Set and Union* idea in graph-like data structures first!

It is widely related to, but not limited to minimum spanning tree algorithms, connectivity judgments, etc. We implement the idea using **Disjoint Set Union (DSU)**.

DSU has two operations: *Union* and *Find*. Every set has a root parent, and commonly we use *path compression* to make the leaves point to the root, keeping the paths as short as possible. **BUT**, a union operation could trigger a situation where one set's root points to another set's root, yet the leaves of the former have not yet updated to point to this new root. Therefore, even when compression is applied to the class, the tree can still have a depth greater than 2, reflecting the concept of a *lazy algorithm*.

```

1  class DSU:
2      def __init__(self, n: int):
3          self.n = n
4          self.par = list(range(n))
5          self.rank = [0] * n
6
7      def find(self, x: int) -> int:
8          if self.par[x] != x:
9              self.par[x] = self.find(self.par[x])
10         return self.par[x]
11
12     def merge(self, x: int, y: int) -> bool:
13         rx = self.find(x)
14         ry = self.find(y)
15         if rx == ry:
16             return False
17         if self.rank[rx] < self.rank[ry]:
18             self.par[rx] = ry
19         elif self.rank[ry] < self.rank[rx]:
20             self.par[ry] = rx
21         else:
22             self.par[rx] = ry
23             self.rank[ry] += 1
24         return True

```

Q8 (Most Stones Removed with Same Row or Column ★★).

On a 2D plane, we place n stones at some integer coordinate points. Each coordinate point may have at most one stone.

A stone can be removed if it shares either the same row or the same column as another stone that has not been removed.

Given an array `stones` of length n where `stones[i] = [xi, yi]` represents the location of the i^{th} stone, return the largest possible number of stones that can be removed.

Translate the problem and we'll see that any stones sharing the same row or column (i.e. a cross pattern) can be reduced to a single stone.

We can store each row in a `rowmap` and each column in a `columnmap`, and if we find a new stone whose row or column is already recorded, we then merge it with the corresponding set and increment our answer by one.

```

1  class Solution:
2      def removeStones(self, stones: list[list[int]]) -> int:
3          n = len(stones)
4          rowmap = {}
5          colmap = {}
6          dsu = DSU(n)
7          ans = 0
8
9          for idx, (x, y) in enumerate(stones):
10             if x not in rowmap:

```

```

11         rowmap[x] = idx
12     else:
13         if dsu.merge(idx, rowmap[x]):
14             ans += 1
15     if y not in colmap:
16         colmap[y] = idx
17     else:
18         if dsu.merge(idx, colmap[y]):
19             ans += 1
20
21     return ans

```

Q9 (Jump DSU ★★★★★).

Given a binary string s , and an integer k . In one operation, you must choose exactly k different indices and flip each '0' to '1' and each '1' to '0'.

Return the minimum number of operations required to make all characters in the string equal to '1'. If it is not possible, return -1.

We can't do this with a simple BFS (would require all permutations).

However, the order is not important in this problem — what about a pruned BFS that only cares about how many '0's are left? This would be $\mathcal{O}(n^2)$ overall, with an $\mathcal{O}(n)$ search space. Please note that even with a visited boolean check, we still consider the overall complexity as $\mathcal{O}(V + E) = \mathcal{O}(V + \mathcal{O}(V)) = \mathcal{O}(V^2)$.

Can we, let's suppose, compress the search space? We don't want to evaluate $\mathcal{O}(n)$ possibilities every single time. Hmm, why not — what if we link every node directly to its successor (or the next valid successor, whenever appropriate), instead of checking that if-statement every time? It acts like arrows on the same level of BFS spanning tree (e.g. there was $1 \rightarrow [2, 5]$ before, so for $2 \rightarrow [3, 6]$ jump $2 \rightarrow 3 \rightarrow 6$ without 4, 5).

Note that if we do this, we **CAN'T** use union by rank — we are defining the root as the next valid successor node! This is the so-called **Jump DSU** trick, suitable for BFS in an ordered search space.

```

1  class DSU:
2      def __init__(self, n: int):
3          self.parent = list(range(n + 3))
4
5      def find(self, i: int) -> int:
6          if self.parent[i] != i:
7              self.parent[i] = self.find(self.parent[i])
8          return self.parent[i]
9
10 class Solution:
11     def minOperations(self, s: str, k: int) -> int:
12         n = len(s)
13         init_z = s.count('0')
14         if init_z == 0:
15             return 0
16         dsu = DSU(n)
17         dist = [-1] * (n + 1)
18         dist[init_z] = 0
19         dsu.parent[init_z] = init_z + 2
20         queue = deque([init_z])

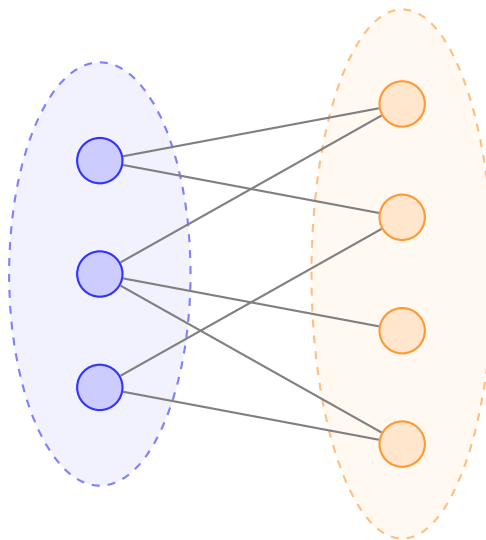
```

```

21     while queue:
22         z = queue.popleft()
23         x_min = max(0, k - n + z)
24         x_max = min(k, z)
25         L = z + k - 2 * x_max
26         R = z + k - 2 * x_min
27         curr = dsu.find(L)
28         while curr <= R:
29             dist[curr] = dist[z] + 1
30             if curr == 0:
31                 return dist[curr]
32             queue.append(curr)
33             nxt = dsu.find(curr + 2)
34             dsu.parent[curr] = nxt
35             curr = nxt
36
37     return -1

```

1.3.2 Bipartite Graphs



See, this is a bipartite graph! Neither partition may contain internal edges.

They are often used in maximum bipartite matching, minimum vertex cover, and maximum independent set problems. They are equivalent by *König's Theorem*:

Let $G = (U \cup V, E)$ be a bipartite graph. Then:

$$|M_{\max}| = |C_{\min}|$$

where $M_{\max}$ is a maximum matching and $C_{\min}$ is a minimum vertex cover.

Q10 (Hungarian Algorithm ★★).

There are m boys and n girls in a class attending an upcoming party.

Given an $m \times n$ integer matrix `grid`, where `grid[i][j]` equals 0 or 1. If `grid[i][j]==1`, then that means the i^{th} boy can invite the j^{th} girl to the party. A boy can invite at most one girl, and a girl can accept at most one invitation from a boy.

Return the maximum possible number of accepted invitations.

The algorithm is straightforward and somewhat *greedy* — try the first available edge for each node in the left partition, and if a subsequent left node's desired right node is already occupied, backtrack to its current match to see if there are other possibilities. That's all there is to the **Hungarian Algorithm**.

```
1 class Solution:
2     def match(self, grid: list[list[int]], l: int) -> bool:
3         for r in range(self.n):
4             if not grid[l][r] or self.visr[r]:
5                 continue
6             self.visr[r] = True
7             if r not in self.mapleft or self.match(grid, self.mapleft[r]):
8                 self.mapleft[r] = l
9             return True
10
11         return False
12
13     def maximumInvitations(self, grid: list[list[int]]) -> int:
14         self.m = len(grid)
15         self.n = len(grid[0])
16         self.mapleft = {}
17         ans = 0
18
19         for i in range(self.m):
20             self.visr = [False] * self.n
21             if self.match(grid, i):
22                 ans += 1
23
24         return ans
```

Q11 (Maximum Students Taking Exam ★★★).

Given a $m \times n$ matrix `seats` that represent seats distributions in a classroom. If a seat is broken, it is denoted by '#' character otherwise it is denoted by a '.' character.

Students can see the answers of those sitting next to the left, right, upper left and upper right, but he cannot see the answers of the student sitting directly in front or behind him. Return the maximum number of students that can take the exam together without any cheating being possible.

This is, at first glance, a maximum independent set problem.

But what are the two partitions?

If we separate the odd columns and the even columns, they form a bipartite graph — neither partition has internal edges, only edges connecting them. Each position can have **AT MOST** one person. Construct the graph by connecting those edges, then run the Hungarian Algorithm!

```

1  class Solution:
2      def check(self, seat: int, field: int) -> bool:
3          sx, sy = self.pos[seat]
4          fx, fy = self.pos[field]
5          return abs(sx - fx) <= 1 and abs(sy - fy) == 1
6
7      def match(self, seats: list[list[str]], l: int) -> bool:
8          for r in self.graph[l]:
9              if self.visr[r]:
10                 continue
11                 self.visr[r] = True
12                 if self.mapl[r] == -1 or self.match(seats, self.mapl[r]):
13                     self.mapl[r] = 1
14                     return True
15
16         return False
17
18     def maxStudents(self, seats: list[list[str]]) -> int:
19         self.m = len(seats)
20         self.n = len(seats[0])
21         self.pos = []
22
23         for i in range(self.m):
24             for j in range(self.n):
25                 if seats[i][j] == '.':
26                     self.pos.append((i, j))
27
28         self.size = len(self.pos)
29         self.graph = [list() for i in range(self.size)]
30         self.mapl = [-1] * self.size
31         self.visr = [False] * self.size
32
33         for i in range(self.size):
34             for j in range(self.size):
35                 if self.pos[i][1] % 2 == 0 and self.pos[j][1] % 2 == 1 and self.
check(i, j):
36                     self.graph[i].append(j)
37
38         cnt = 0
39
40         for i in range(self.size):
41             if self.pos[i][1] % 2 == 1:
42                 continue
43                 self.visr = [False] * self.size
44                 if self.match(seats, i):
45                     cnt += 1
46
47         return self.size - cnt

```

Q12 (Muddy Fields ★★★★★).

Given an $m \times n$ grid consisting of muddy cells '*' and grassy cells '.', find the minimum number of wooden boards needed to cover all muddy cells.

Rules:

- Each board has a width of 1 and any integer length.
- Boards can only be placed horizontally or vertically.
- Boards may overlap each other.
- Boards must not cover any grassy cells.

This even looks NP-ish. But stay calm — I put this question in this section, which implies it can be solved in polynomial time using a minimum vertex cover algorithm. Fortunately, we can!

Treat every muddy hole as an edge, and the two partitions as the horizontal and vertical wooden boards. After that, every muddy hole serves as an edge connecting a horizontal board to a vertical board. Think about a continuous line of mud that only has a width of 1: it would represent just one vertical board, but multiple length-1 horizontal boards. After that, the Hungarian Algorithm is all you need!

```
1  class Solution:
2      def match(self, u: int) -> bool:
3          for v in self.graph[u]:
4              if self.vis[v]:
5                  continue
6              self.vis[v] = True
7              if self.matchV[v] == -1 or self.match(self.matchV[v]):
8                  self.matchV[v] = u
9                  return True
10
11     return False
12
13     def minBoards(self, grid: list[str]) -> int:
14         if not grid or not grid[0]:
15             return 0
16         R = len(grid)
17         C = len(grid[0])
18         rowID = [[-1] * C for i in range(R)]
19         colID = [[-1] * C for i in range(R)]
20         uCnt = 0
21         vCnt = 0
22
23         for r in range(R):
24             inSegment = False
25             for c in range(C):
26                 if grid[r][c] == '*':
27                     rowID[r][c] = uCnt
28                     inSegment = True
29                 elif inSegment:
30                     uCnt += 1
31                     inSegment = False
32             if inSegment:
33                 uCnt += 1
34
35         for c in range(C):
```

```

36         inSegment = False
37         for r in range(R):
38             if grid[r][c] == '*':
39                 colID[r][c] = vCnt
40                 inSegment = True
41             elif inSegment:
42                 vCnt += 1
43                 inSegment = False
44         if inSegment:
45             vCnt += 1
46
47     self.graph = [list() for i in range(uCnt)]
48
49     for r in range(R):
50         for c in range(C):
51             if grid[r][c] == '*':
52                 self.graph[rowID[r][c]].append(colID[r][c])
53
54     self.matchV = [-1] * vCnt
55     ans = 0
56
57     for u in range(uCnt):
58         self.vis = [False] * vCnt
59         if self.match(u):
60             ans += 1
61
62     return ans

```

Note: Beyond the Hungarian Algorithm, a more general approach is *maximum flow* — but I’m not going to go too deeply into this topic — if you’re interested, please take a look at *Dinic’s Algorithm*!

2 Algorithms

2.1 Handy Tricks

2.1.1 Prime Sieve

Euler Sieve is the most popular way to do this, and sieving is one of the first topics in number theory. How do we do it in $\mathcal{O}(n)$? Or in other words, how do we avoid counting the same number twice?

Intuitively, think of every composite number as a prime times something else. We can scan through all numbers and multiply each prime that is less than or equal to them, making sure the product is no larger than n .

```
1 for i in range(2, n + 1):
2     for p in primes:
3         if i * p > n:
4             break
5         is_prime[i * p] = False
```

However, this is not $\mathcal{O}(n)$ (think about 4×3 and 6×2) — anyways, we've invented *Eratosthenes Sieve*! Actually, the Euler Sieve merely adds **ONE** conditional line to handle this rule, that is, to skip those divisibility conditions.

```
1 res = 1
2 while b > 0:
3     if b % 2 == 1:
4         res = res * a
5     a = a * a
6     b = b // 2
```

2.1.2 KMP Algorithm

Knuth-Morris-Pratt Algorithm is the classical deterministic way to match strings, though nowadays search engines are not likely to use it. It is a method for matching an $\mathcal{O}(m)$ target string within a large $\mathcal{O}(n)$ source text and returning its location in $\mathcal{O}(m + n)$ time. It is the optimal lower bound.

Let's think about a middle step — suppose we are trying to match ABAC using pointer p1 within the string ABABAC using pointer p2. Right now, p1 is at C and p2 is at the second B. It looks like they don't match, but we don't reset p2 to the first B this time. Instead, we keep p2 stable and move p1 to the nearest prefix re-match (p1 can even jump further back after this, if needed, which is doable in $\mathcal{O}(m)$ time). So p1 becomes B again, and then the whole string matches!

Technically, whenever we find a letter that does not match, p1 jumps by checking the longest, then the second longest, maximum common prefix-suffix of the substring before it. Note that this can jump at most $\mathcal{O}(n)$ times in total, which is exactly the intuition behind *Amortized Analysis*.

```
1 j = 0
2 for i in range(1, m):
3     while j > 0 and pattern[i] != pattern[j]:
4         j = next_arr[j - 1]
5     if pattern[i] == pattern[j]:
6         j += 1
7     next_arr[i] = j
```

```

8  j = 0
9  for i in range(n):
10     while j > 0 and text[i] != pattern[j]:
11         j = next_arr[j - 1]
12     if text[i] == pattern[j]:
13         j += 1
14     if j == m:
15         return i - m + 1

```

2.1.3 Quick Power

Quick Power is another small and handy trick that gives us some food for thought.

The power 2^{10} is the same as $2^{(10^{10})^2} = 2^8 \times 2^2$, we only need $\mathcal{O}(\log n)$ time to calculate $2^1, 2^2, 2^4, \dots$, and that's enough!

```

1  res = 1
2  while b > 0:
3      if b % 2 == 1:
4          res = res * a
5      a = a * a
6      b = b // 2

```

What if the exponent is unprecedentedly large? Check *Extended Euler's Theorem* or *Lightspeed Power* in $\mathcal{O}(1)$ time!

Q13 (Lightspeed Power ★★★).

Given fixed base a and fixed modulo mod , compute $a^b \pmod{\text{mod}}$ for 10^7 queries, where the exponent b is within a 32-bit integer range.

The core idea is:

$$a^b = a^{x \times S} \times a^y$$

where we often choose $S = \sqrt{n}$, which in this case is 2^{16} .

```

1  pow1 = [0] * 65536
2  pow2 = [0] * 65536
3
4  def preproc():
5      pow1[0] = 1
6      pow2[0] = 1
7
8      for i in range(1, 65536):
9          pow1[i] = (pow1[i - 1] * a) % mod
10
11     pow65536 = (pow1[65535] * a) % mod
12
13     for i in range(1, 65536):
14         pow2[i] = (pow2[i - 1] * pow65536) % mod

```

```

15 def query(pows):
16     return (pow1[pows & 65535] * pow2[pows >> 16]) % mod

```

2.2 Search

2.2.1 DFS

Depth-First uses **LIFO**, it has the following template:

```

1 def dfs(node):
2     if node in visited:
3         return
4     visited.add(node)
5     for nxt in node.nexts:
6         dfs(nxt)

```

Set `visited` ensures that we never visit the same node twice. This runs in $\mathcal{O}(V + E)$ time.

Q14 (Sum of Root To Leaf ★).

Given the root of a binary tree where each node has a value 0 or 1. Each root-to-leaf path represents a binary number starting with the most significant bit.

- For example, if the path is 0->1->1->0->1, then this could represent 01101 in binary, which is 13.

For all leaves in the tree, consider the numbers represented by the path from the root to that leaf. Return the sum of these numbers.

Think of the data stream as starting from the root (top to bottom), passing to each child, and concluding the answer after all paths have been processed.

```

1 class Solution:
2     def sumRootToLeaf(self, root: Optional[TreeNode]) -> int:
3         def dfs(node, val) -> int:
4             if not node: return 0
5             val = (val << 1) | node.val
6             if not node.left and not node.right: return val
7             return dfs(node.left, val) + dfs(node.right, val)
8
9         return dfs(root, 0)

```

Q15 (Lowest Common Ancestor ★★).

Given the root of a binary tree, return the lowest common ancestor of its deepest leaves.

We need to know the depth of every node to determine the LCA. So, perform a bottom-up DFS this time. It has a *Tree Divide and Conquer* idea behind it.

```

1 class Solution:
2     def lcaDeepestLeaves(self, root: Optional[TreeNode]) -> Optional[TreeNode]:
3         def dfs(node, depth) -> tuple[TreeNode, int]:

```

```

4         ldepth = depth
5         rdepth = depth
6         if node.left:
7             lnode, ldepth = dfs(node.left, depth + 1)
8         if node.right:
9             rnode, rdepth = dfs(node.right, depth + 1)
10        if ldepth > rdepth:
11            return lnode, ldepth
12        elif ldepth < rdepth:
13            return rnode, rdepth
14        else:
15            return node, ldepth
16
17    ans, _ = dfs(root, 0)
18    return ans

```

Note: If we are supposed to find all possible paths, DFS can perform a special technique: *backtracking*. This runs in exponential time.

```

1  def dfs(node):
2      if node in visited:
3          return
4      visited.add(node)
5      for nxt in node.nexts:
6          dfs(nxt)
7      visited.remove(node)

```

2.2.2 BFS

Breadth-First uses **FIFO**, it has the following template:

```

1  while queue:
2      node = queue.popleft()
3      for nxt in node.nexts:
4          if nxt not in visited:
5              visited.add(nxt)
6              queue.append(nxt)

```

This also runs in $\mathcal{O}(V + E)$ time, with higher memory consumption than DFS.

For interval search spaces, the common BFS is $\mathcal{O}(n^2)$, but this can be improved to $\mathcal{O}(n \log n)$ using a red-black tree or $\mathcal{O}(n\alpha(n))$ using Jump DSU (recommended, see an earlier question).

2.2.3 Binary Search

Python provides binary search functions:

```

1  lower_bound = bisect.bisect_left(arr, target)
2  upper_bound = bisect.bisect_right(arr, target)

```

If we need to do more than simply find the bisect position, we'll need to write binary search by hand.

Q16 (Eating Bananas ★).

There are n piles of bananas, the i^{th} pile has `piles[i]` bananas. The guards have gone and will come back in h hours.

The monkey can decide its bananas-per-hour eating speed of k . Each hour, it chooses some pile of bananas and eats k bananas from that pile. If the pile has less than k bananas, it eats all of them instead and will not eat any more bananas during this hour.

The monkey likes to eat slowly but still wants to finish eating all the bananas before the guards return. Return the minimum integer k such that the monkey can eat all the bananas within h hours.

```
1  class Solution:
2      def minEatingSpeed(self, piles: list[int], h: int) -> int:
3          piles.sort()
4          n = len(piles)
5          l = 1
6          r = piles[n - 1]
7
8          while l < r:
9              mid = l + (r - l) // 2
10             if self.check(piles, h, mid):
11                 r = mid
12             else:
13                 l = mid + 1
14
15         return r
16
17     def check(self, piles: list[int], h: int, k: int) -> bool:
18         time = 0
19
20         for i in piles:
21             time += math.ceil(i / k)
22
23         if time <= h:
24             return True
25         return False
```

2.3 Divide and Conquer

There is another special word for **Divide and Conquer** — *non-overlapping substructures*.

Segment trees (see before) and Merge Sort are typical examples of **Classical D&C**. Besides these, **Tree D&C** is another big branch (known as *Centroid Decomposition* and *Edge Decomposition*).

Q17 (2-Merge Sort ★).

Given the heads of two sorted linked lists `list1` and `list2`.

Merge the two lists into one sorted list. The list should be made by splicing together the nodes of the first two lists.

Consider two pointers `p1` and `p2` for `list1` and `list2`. Since they are pre-sorted, the only work left is to compare `*p1` and `*p2` each time before deciding which pointer to move right!

```

1  class Solution:
2      def mergeTwoLists(self, list1: Optional[ListNode], list2: Optional[ListNode])
   -> Optional[ListNode]:
3          root = ListNode(-1)
4          current = root
5
6          while list1 and list2:
7              if list1.val < list2.val:
8                  current.next = list1
9                  list1 = list1.next
10             else:
11                 current.next = list2
12                 list2 = list2.next
13             current = current.next
14
15         if list1:
16             current.next = list1
17         if list2:
18             current.next = list2
19         return root.next

```

Q18 (k-Merge Sort ★★).

Given an array of k linked-lists `lists`, each linked-list is sorted in ascending order. Merge all the linked-lists into one sorted linked-list and return it.

If we follow the same procedure, the difference here is how to compare k numbers to decide which pointer to move (which could easily go up to $\mathcal{O}(nk)$ time)! So, time to use a min-heap again, with a time complexity of $\mathcal{O}(n \log k)$.

```

1  class Solution:
2      def mergeKLists(self, lists: List[Optional[ListNode]]) -> Optional[ListNode]:
3          root = ListNode(-1)
4          current = root
5          heap = []
6
7          for i, node in enumerate(lists):
8              if node:
9                  heapq.heappush(heap, (node.val, i, node))
10
11         while heap:
12             val, i, node = heapq.heappop(heap)
13             current.next = node
14             current = current.next
15             if node.next:
16                 heapq.heappush(heap, (node.next.val, i, node.next))
17
18         return root.next

```

2.4 Dynamic Programming

There is another special word for **Dynamic Programming** — *overlapping substructures*.

Note: We are mostly implementing DP in a *bottom-up* way rather than *top-down / memoization* below. I like DP ideas very much at least when writing this.

Let's explore some classical examples of **Linear DP** at the beginning.

a) Linear DP

Longest Increasing Subsequence — given a sequence L , find the length of the longest subsequence in which all elements are strictly increasing:

$$f(i) = \begin{cases} 1, & n(i) \leq n(i-k) \\ \max_k f(i-k) + 1, & n(i) > n(i-k) \end{cases}$$

where $f(i)$ represents the length of the longest increasing subsequence ending at index i .

Longest Increasing Subarray — the difference is subtle again:

$$f(i) = \begin{cases} 1, & n(i) \leq n(i-1) \\ f(i-1) + 1, & n(i) > n(i-1) \end{cases}$$

Longest Common Subsequence — for sequence p with length m and sequence q with length n :

$$f(i, j) = \begin{cases} \max(f(i-1, j), f(i, j-1)), & p(i) \neq q(j) \\ f(i-1, j-1) + 1, & p(i) = q(j) \end{cases}$$

where $f(i, j)$ represents the length of the longest common subsequence between $p[0\dots i]$ and $q[0\dots j]$.

Maximum Subarray Sum — given an integer array a , find the contiguous subarray with the largest possible sum and return that sum:

$$f(i) = \max(f(i-1) + a(i), a(i))$$

where i represents the last index of the subarray with the maximum sum ending at i . This is also known as *Kadane's Algorithm*.

Gap-Taxed Largest Subsequence — we introduce a penalty when there is a gap between two selected elements. For example, define the tax as:

$$\text{penalty}(i, j) = (i - j - 1)^2$$

then the state transition becomes:

$$f(i) = \max_{j < i} (a(i), f(j) + a(i) - \text{penalty}(i, j))$$

where $f(i)$ represents the maximum score of a gap-taxed subsequence ending at index i . This can be viewed as a generalization of *Kadane's Algorithm*.

Knapsack DP is a huge branch of Linear DP. Now let's take a closer look at it.

0/1 Knapsack — given a set of n items, where each item has a weight w_i and a value v_i , the goal is to select the most valuable subset of items that fits within a knapsack of capacity W .

This is also known as the *yes/no choice* pattern: for each item, we either take it or leave it.

The substructure is built at item step i :

$$f(i, W) = \max(f(i-1, W), f(i-1, W - w(i)) + v(i))$$

where $f(0, W) = 0$ and $i = 1, 2, \dots, n$.

Fractional Knapsack — we can take fractional items, it can be solved by a *Greedy Algorithm*.

Bounded/Unbounded Knapsack — the tiny difference here is whether each item has a limited number of copies. Bounded means each item has a fixed limit (also called **Multiple Knapsack**), while unbounded means we can take unlimited copies (also called **Complete Knapsack**).

For the unbounded case:

$$f(i, W) = \max(f(i-1, W), f(i, W - w(i)) + v(i))$$

where i represents the current item type.

For the bounded case:

$$f(i, W) = \max_{0 \leq k \leq k_i} (f(i-1, W - kw(i)) + kv(i))$$

where k_i is the maximum number of copies allowed for item i .

Multi-dimensional Knapsack — the formulation is:

$$\max \sum_{i=1}^n v_i x_i \quad \text{S.T.} \quad \sum_{i=1}^n w_{ij} x_i \leq W_j$$

where $x_i \in \{0, 1\}$.

The recurrence is quite similar (but in vectors):

$$f(i, \mathbf{W}) = \max(f(i-1, \mathbf{W}), f(i-1, \mathbf{W} - \mathbf{w}(i)) + v(i))$$

this is getting insanely expensive as the dimension grows, so it is often combined with other techniques for pruning.

The complexity above is determined by the size of the DP matrix, which is $\mathcal{O}(nW)$ for the one-dimensional case, or $\mathcal{O}(nW_1 \cdots W_d)$ for the multi-dimensional case.

Note: I believe this is enough for Linear DP exercises, and please take some time to digest them!

And one more thing — AI is everywhere, and I feel it is important to show DP in **Backpropagation**:

You are given a multilayer neural network with M layers and N neurons per layer.

For each layer i :

- a_j^i : the output representation of neuron j in layer i
- $w_{j,k}^i$: the weight connecting neuron j in layer i to neuron k in layer $i + 1$
- $f(\cdot)$: the activation function
- ℓ : the final loss

The task is to compute:

$$\frac{\partial \ell}{\partial w_{j,k}^i}$$

in $\mathcal{O}(MN^2)$ time.

The key idea is simple — we reuse intermediate gradients through dynamic programming:

$$\frac{\partial z}{\partial a_j^i} = \sum_{k=1}^N \frac{\partial z}{\partial a_k^{i+1}} \cdot \frac{\partial a_k^{i+1}}{\partial a_j^i}$$

$$\frac{\partial z}{\partial w_{j,k}^i} = \sum_{m=1}^N \frac{\partial z}{\partial a_m^i} \cdot \frac{\partial a_m^i}{\partial w_{j,k}^i}$$

where i denotes the i^{th} layer, a^i represents the hidden representation of layer i , $w_{j,k}^i$ is the $(j,k)^{\text{th}}$ entry of the weight matrix at layer i , and z represents the loss function.

b) Interval DP

Let's start with my favorite classical palindrome problems.

Q19 (Longest Palindromic Subsequence ★★).

Given a string s , find the longest palindromic subsequence's length in s .

A subsequence is a sequence that can be derived from another sequence by deleting some or no elements without changing the order of the remaining elements.

For a *palindrome*, the intuition is to consider two directions, i.e. going left and going right.

So use a 2D DP array to store the lengths, where the two dimensions represent the start and end indices of the string respectively. The state transition is, if the characters at both ends are equal, $\text{dp}[i][j] = \text{dp}[i+1][j-1] + 2$, plus the boundary condition for $j-i \leq 1$.

Otherwise, run $\text{dp}[i][j] = \max(\text{dp}[i+1][j], \text{dp}[i][j-1])$.

```

1  class Solution:
2      def longestPalindromeSubseq(self, s: str) -> int:
3          n = len(s)
4          dp = [[0] * n for _ in range(n)]
5
6          for i in range(n - 1, -1, -1):
7              dp[i][i] = 1
8              for j in range(i + 1, n):
9                  if s[i] == s[j]:

```

```

10         dp[i][j] = dp[i + 1][j - 1] + 2
11     else:
12         dp[i][j] = max(dp[i + 1][j], dp[i][j - 1])
13
14     return dp[0][n - 1]

```

Hold on! Think again about a subtle property here. We know that a subsequence doesn't have to be contiguous, so the state transition follows an additive rule, i.e. an inner result can contribute directly to its enclosing interval. But what if, the subsequence has to be contiguous? What happens then?

Q20 (Longest Palindromic Substring ★★).

Given a string s , return the longest palindromic substring in s .

The additive rule doesn't apply here. We can't simply use $dp[i][j] = dp[i+1][j-1] + 2$ for the state transition anymore. So what's left? We can only mark each substring with a boolean indicating whether it's a palindrome, and maintain a global counter as we go.

```

1  class Solution:
2      def longestPalindrome(self, s: str) -> str:
3          n = len(s)
4          dp = [[False] * n for _ in range(n)]
5          ans, idx = 0, -1
6
7          for l in range(1, n + 1):
8              for i in range(n - l + 1):
9                  j = i + l - 1
10                 if j >= n: break
11                 if s[i] == s[j]:
12                     if j - i <= 1: dp[i][j] = True
13                     else: dp[i][j] = dp[i + 1][j - 1]
14                 if dp[i][j] and ans < j - i + 1:
15                     ans = j - i + 1
16                     idx = i
17
18     return s[idx:idx + ans]

```

We might find it often to define i and j as the start and end indices of a string. But note that **NOT** all 2D arrays are **Interval DPs**, since they should have two indices representing directions on the same measure.

Q21 (Zuma ★★★).

Genos recently installed the game Zuma on his phone. In Zuma there exists a line of n gemstones, the i^{th} of which has color c_i . The goal of the game is to destroy all the gemstones in the line as quickly as possible. In one second, Genos is able to choose exactly one continuous substring of colored gemstones that is a palindrome and remove it from the line. After the substring is removed, the remaining gemstones shift to form a solid line again. What is the minimum number of seconds needed to destroy the entire line?

Can we use a *Greedy Algorithm* here? Unfortunately, it doesn't always work (think about why $1, 2, 1, 1, 3, 1$). By this counterexample, the time complexity rises to $\mathcal{O}(n^3)$:

$$f(i, j) = \begin{cases} \min \begin{cases} f(i+1, j-1) \\ \min_k (f(i, k) + f(k+1, j)) \end{cases} & \text{if } \text{colors}[i] == \text{colors}[j] \\ \min_k (f(i, k) + f(k+1, j)), & \text{if } \text{colors}[i] \neq \text{colors}[j] \end{cases}$$

which has a shrinking algorithm and a splitting algorithm.

```

1  class Solution:
2      def minimumMoves(self, colors: List[int]) -> int:
3          n = len(colors)
4          if n == 0:
5              return 0
6          dp = [[float('inf')] * n for _ in range(n)]
7
8          for length in range(1, n + 1):
9              for i in range(n):
10                 j = i + length - 1
11                 if j >= n:
12                     break
13                 if i == j:
14                     dp[i][j] = 1
15                     continue
16                 if colors[i] == colors[j]:
17                     if j - i == 1:
18                         dp[i][j] = 1
19                     else:
20                         dp[i][j] = dp[i + 1][j - 1]
21                 for k in range(i, j):
22                     dp[i][j] = min(dp[i][j], dp[i][k] + dp[k + 1][j])
23
24         return dp[0][n - 1]
```

c) Tree DP

As the name suggests, **Tree DP** is simply DP on a tree.

Q22 (House Robber ★★).

The thief has found himself a new place for his thievery again. There is only one entrance to this area, called `root`.

Besides the `root`, each house has one and only one parent house. After a tour, the smart thief realized that all houses in this place form a binary tree. It will automatically contact the police if two directly-linked houses were broken into on the same night.

Given the `root` of the binary tree, return the maximum amount of money the thief can rob without alerting the police.

This problem is very suitable for overlapping substructures.

Think about:

$$f(u) = \max \begin{cases} \text{val}(u) + \sum_{v \in \text{grandchild}(u)} f(v), & (\text{if steal } u) \\ \sum_{v \in \text{child}(u)} f(v), & (\text{if not steal } u). \end{cases}$$

```

1  class Solution:
2      def rob(self, root: Optional[TreeNode]) -> int:
3          def dp(node) -> tuple[int, int]:
4              if not node:
5                  return (0, 0)
6              ly, ln = dp(node.left)
7              ry, rn = dp(node.right)
8              return (node.val + ln + rn, max(ly, ln) + max(ry, rn))
9
10         ry, rn = dp(root)
11         return max(ry, rn)

```

Q23 (Binary Tree Maximum Path Sum ★★).

A path in a binary tree is a sequence of nodes where each pair of adjacent nodes in the sequence has an edge connecting them. A node can only appear in the sequence at most once. Note that the path does not need to pass through the root.

The path sum of a path is the sum of the node's values in the path.

Given the root of a binary tree, return the maximum path sum of any non-empty path.

The question needs a translation before solving. Think about it: every path should have one and only one root, so let's store the maximum straight path sum for every node. The answer would be obtained by adding two subtrees together and comparing them:

$$f(u) = \max_{v \in \text{child}(u)} g(v).$$

```

1  class Solution:
2      def maxPathSum(self, root: Optional[TreeNode]) -> int:
3          ans = -1000
4
5          def dp(node) -> int:
6              nonlocal ans
7              if not node:
8                  return 0
9              l = dp(node.left)
10             r = dp(node.right)
11             ans = max(ans, node.val + max(0, l) + max(0, r))
12             return node.val + max(0, l, r)
13
14         dp(root)
15         return ans

```

d) Bitmask DP

Bitmask DP is a special branch of DP. Its core idea is to use a bit number to represent a state.

Q24 (Held-Karp Algorithm ★★★★★).

Given an array of strings `words`, return the smallest string that contains each string in `words` as a substring. If there are multiple valid strings of the smallest length, return any of them. May assume that no string in `words` is a substring of another string in `words`.

It's common for a word's suffix to overlap with another word's prefix — how do we count this (or modelize this)? Here comes the first Aha moment: define $w(i, j)$ to represent the overlap between the i -suffix and the j -prefix (the graph is directional).

Now the problem is translated into: for a weighted directed graph, how do we find the shortest path that connects all of them?

The first intuition is to split any connected edge in S and add a $v \notin S$ to insert into one of them, before deleting the longest edge. However, it's easy to see that this will result in disconnected components. We need to make sure that the previous path wouldn't be cut!

Here comes the second Aha moment by applying Bitmask DP:

$$f(S \cup \{v\}, v) = f(S, u) + w(u, v)$$

where u is the last node of S .

The second step is also called the *Travelling Salesman Problem (TSP)*, which is one of the most famous NP-complete problems. We have $\mathcal{O}(2^n)$ for enumerating S , and $\mathcal{O}(n)$ for checking whether v is outside S , and we count $\mathcal{O}(n)$ times for every $u \in S$, so the complexity is $\mathcal{O}(n^2 2^n)$.

```
1  class Solution:
2      def shortestSuperstring(self, words: List[str]) -> str:
3          n = len(words)
4          w = [[0] * n for _ in range(n)]
5
6          for i in range(n):
7              for j in range(n):
8                  if i != j:
9                      max_overlap = 0
10                     min_len = min(len(words[i]), len(words[j]))
11                     for k in range(min_len, 0, -1):
12                         if words[i].endswith(words[j][:k]):
13                             max_overlap = k
14                             break
15                     w[i][j] = len(words[j]) - max_overlap
16
17         dp = [[float('inf')] * n for _ in range(1 << n)]
18         parent = [[-1] * n for _ in range(1 << n)]
19
20         for i in range(n):
21             dp[1 << i][i] = len(words[i])
22
23         for S in range(1, 1 << n):
```

```

24         for i in range(n):
25             if not (S & (1 << i)): continue
26             for j in range(n):
27                 if S & (1 << j): continue
28                 next_S = S | (1 << j)
29                 if dp[S][i] + w[i][j] < dp[next_S][j]:
30                     dp[next_S][j] = dp[S][i] + w[i][j]
31                     parent[next_S][j] = i
32
33     final_state = (1 << n) - 1
34     min_len = float('inf')
35     last_node = -1
36
37     for i in range(n):
38         if dp[final_state][i] < min_len:
39             min_len = dp[final_state][i]
40             last_node = i
41     path = []
42     curr_state = final_state
43     curr_node = last_node
44
45     while curr_node != -1:
46         path.append(curr_node)
47         prev_node = parent[curr_state][curr_node]
48         curr_state ^= (1 << curr_node)
49         curr_node = prev_node
50
51     path.reverse()
52     ans = words[path[0]]
53
54     for i in range(1, len(path)):
55         prev_word = path[i-1]
56         curr_word = path[i]
57         cost = w[prev_word][curr_word]
58         if cost > 0:
59             ans += words[curr_word][-cost:]
60
61     return ans

```

2.5 Graph Theory

2.5.1 Minimum Spanning Tree

Minimum Spanning Tree (MST) algorithms are *Greedy Algorithms*, based on the *Cut Property*:

Let $G = (V, E)$ be a connected undirected graph with an edge weight function w , and let $S \subset V$ be any non-empty proper subset of vertices. Then:

$$e = \arg \min_{e' \in \delta(S)} w(e') \implies e \in T$$

where $\delta(S)$ is the set of edges crossing the cut $(S, V \setminus S)$, and T is some minimum spanning tree of G .

a) Kruskal's Algorithm

We use DSUs to implement Kruskal's Algorithm (and treat every node as a disjoint set at the beginning). For every edge connecting two different sets, pick it if it has the minimum weight, and iterate. The time complexity is $\mathcal{O}(E \log E)$. Looks like we are encountering more named algorithms!

```
1  def kruskal(all_nodes, all_edges):
2      mst_edges = []
3
4      for node in all_nodes:
5          parent[node] = node
6
7      sorted_edges = sorted(all_edges, key=lambda e: e.weight)
8
9      for edge in sorted_edges:
10         if union(edge.u, edge.v):
11             mst_edges.append(edge)
12             if len(mst_edges) == len(all_nodes) - 1:
13                 break
14
15     return mst_edges
```

b) Prim's Algorithm

Prim's Algorithm is based on maintaining $V \setminus S$, and each time picking the smallest edge between S and $V \setminus S$ to join in, often using a heap to maintain the order. The time complexity is $\mathcal{O}((V + E) \log V)$.

```
1  def prim(start_node):
2      visited.add(start_node)
3      for edge in start_node.edges:
4          heapq.heappush(min_heap, (edge.weight, edge.to_node, edge))
5
6      while min_heap:
7          weight, next_node, current_edge = heapq.heappop(min_heap)
8          if next_node in visited:
9              continue
10         visited.add(next_node)
11         mst_edges.append(current_edge)
12         for edge in next_node.edges:
13             if edge.to_node not in visited:
14                 heapq.heappush(min_heap, (edge.weight, edge.to_node, edge))
15
16     return mst_edges
```

2.5.2 Single Source Shortest Path

Single Source Shortest Path (SSSP) is probably the most classical problem in graph theory.

a) Floyd-Warshall Algorithm

Technically, Floyd-Warshall Algorithm is not single-sourced, it calculates all-pairs shortest paths using:

$$f(i, j) = \min(f(i, j), f(i, k) + f(k, j)).$$

The time complexity is $\mathcal{O}(V^3)$.

```
1 def floyd_warshall(all_nodes, dist_matrix):
2     for k in all_nodes:
3         for i in all_nodes:
4             for j in all_nodes:
5                 if dist_matrix[i][k] + dist_matrix[k][j] < dist_matrix[i][j]:
6                     dist_matrix[i][j] = dist_matrix[i][k] + dist_matrix[k][j]
```

b) Dijkstra's Algorithm

Dijkstra's Algorithm is a *Greedy Algorithm* too (very much like Prim's Algorithm). The core idea is maintaining a visited set, and each time selecting the shortest edge between the set and the unvisited nodes (often using a min-heap here) and then we don't check it anymore. The time complexity is also $\mathcal{O}((V + E) \log V)$.

Note: It only works for graphs with non-negative edge weights.

```
1 def dijkstra(start_node, all_nodes):
2     for node in all_nodes:
3         dist[node] = float('inf')
4     dist[start_node] = 0
5     min_heap = [(0, start_node)]
6
7     while min_heap:
8         current_dist, current_node = heapq.heappop(min_heap)
9         if current_dist > dist[current_node]:
10            continue
11        for edge in current_node.edges:
12            neighbor = edge.to_node
13            new_path_dist = current_dist + edge.weight
14            if new_path_dist < dist[neighbor]:
15                dist[neighbor] = new_path_dist
16                heapq.heappush(min_heap, (new_path_dist, neighbor))
```

c) Bellman-Ford Algorithm

Bellman-Ford Algorithm could be used for finding negative-weight shortest paths. The time complexity is $\mathcal{O}(V \times E)$ based on the following *Graph DP* transition:

$$f(v) = \min(f(v), w(u, v) + f(u)).$$

```
1 def bellman_ford(start_node, all_nodes, all_edges):
2     for node in all_nodes:
3         dist[node] = float('inf')
4
5     dist[start_node] = 0
6     V = len(all_nodes)
7
8     for _ in range(V - 1):
9         updated = False
10        for edge in all_edges:
11            u = edge.from_node
```

```

12         v = edge.to_node
13         if dist[u] + edge.weight < dist[v]:
14             dist[v] = dist[u] + edge.weight
15             updated = True
16         if not updated:
17             break
18
19     return dist

```

2.5.3 Strongly Connected Components

Strongly Connected Components (SCC) are defined as components where every node can reach every other node in a directed graph. They are used to convert a directed graph into a *Directed Acyclic Graph (DAG)*.

a) Tarjan's Algorithm

The key idea of Tarjan's Algorithm is the DFS number and the low-link value, based on three node states: unvisited, visited and in the stack, and visited but not in the stack. The nodes popped out at the same time form one complete SCC. The time complexity is $\mathcal{O}(V + E)$.

Note: For undirected graphs, DFS is enough for finding connected components. There is also Tarjan's Algorithm, which is used for finding *Biconnected Components*.

```

1  def dfs(node):
2      global time
3      dfn[node] = time
4      low[node] = time
5      time += 1
6      stack.append(node)
7      on_stack.add(node)
8
9      for nxt in node.nexts:
10         if nxt not in dfn:
11             dfs(nxt)
12             low[node] = min(low[node], low[nxt])
13         elif nxt in on_stack:
14             low[node] = min(low[node], dfn[nxt])
15     if low[node] == dfn[node]:
16         scc = []
17         while True:
18             top = stack.pop()
19             on_stack.remove(top)
20             scc.append(top)
21             if top == node:
22                 break
23         all_sccs.append(scc)
24
25 def find_all_sccs(all_nodes):
26     for node in all_nodes:
27         if node not in dfn:
28             dfs(node)
29     return all_sccs

```

b) Kosaraju's Algorithm

Kosaraju's Algorithm applies a simple rule: the transpose of a directed graph does not change its SCCs (and only its SCCs). Note that the *topological order* of the SCC stack never changes, even if the starting point of the DFS changes.

So — if we perform a reverse DFS from the top of the topological stack, it ensures that each search really finds one SCC (instead of the circumstance where nodes that are not an SCC in the original graph appear to be connected in the transposed graph). The time complexity is $\mathcal{O}(V + E)$.

This idea is based on *Topological Sort*.

```
1  def dfs_pass1(node):
2      if node in visited:
3          return
4      visited.add(node)
5      for nxt in node.nexts:
6          dfs_pass1(nxt)
7      stack.append(node)
8
9  def dfs_pass2(node):
10     if node in visited:
11         return
12     visited.add(node)
13     current_scc.append(node)
14     for nxt in node.reversed_nexts:
15         dfs_pass2(nxt)
16
17  def find_all_sccs(all_nodes):
18     global current_scc
19
20     for node in all_nodes:
21         if node not in visited:
22             dfs_pass1(node)
23
24     visited.clear()
25
26     while stack:
27         node = stack.pop()
28         if node not in visited:
29             current_scc = []
30             dfs_pass2(node)
31             all_sccs.append(current_scc)
32
33     return all_sccs
```

2.5.4 Topological Sort

Topological Sort is only defined on DAGs. This can also be used to check whether a cycle exists.

a) DFS

The first DFS pass in Kosaraju's Algorithm is a Topological Sort. The most magical thing here is that a single DFS stack contains all the information needed for the topological order — source nodes must be on the

top of the stack, and sink nodes must be at the bottom of the hierarchy. The time complexity is $\mathcal{O}(V + E)$.

```
1 def dfs_topo(node):
2     if node in visiting:
3         return False
4     if node in visited:
5         return True
6     visiting.add(node)
7
8     for nxt in node.nexts:
9         if not dfs_topo(nxt):
10            return False
11
12     visiting.remove(node)
13     visited.add(node)
14     stack.append(node)
15     return True
16
17 def dfs_topological_sort(all_nodes):
18     for node in all_nodes:
19         if node not in visited:
20             if not dfs_topo(node):
21                 return None
22
23     return stack[::-1]
```

b) Kahn's Algorithm

Kahn's Algorithm is using a queue to store all 0-in-degree nodes. For each turn, remove them and update the in-degrees, resulting in the topological order (very natural from a human perspective). The time complexity is $\mathcal{O}(V + E)$.

```
1 def kahn(all_nodes):
2     in_degree = {node: 0 for node in all_nodes}
3     for node in all_nodes:
4         for nxt in node.nexts:
5             in_degree[nxt] += 1
6
7     queue = deque([node for node in all_nodes if in_degree[node] == 0])
8     topo_order = []
9
10    while queue:
11        current = queue.popleft()
12        topo_order.append(current)
13        for nxt in current.nexts:
14            in_degree[nxt] -= 1
15            if in_degree[nxt] == 0:
16                queue.append(nxt)
17
18    if len(topo_order) == len(all_nodes):
19        return topo_order
20    else:
21        return None
```
